

Conclusion:

The **Implement Client Script & UI Policy (Incident)** micro project demonstrates how ServiceNow can be customized to improve the functionality, usability, and data accuracy of Incident forms. The project combines **Client Scripts** and **UI Policies** to create a dynamic and controlled Incident Management environment.

In this project, different types of Client Scripts such as **onChange**, **onSubmit**, and **onCellEdit** are used to control how users interact with Incident records. The **onChange Client Script** can automatically update fields based on changes made by the user. For example, when the **Impact** of an Incident is changed to **High**, the system can automatically set the **Urgency** to High. This reduces manual effort and helps maintain consistent Incident information.

The **onSubmit Client Script** provides an additional level of validation before an Incident is saved. For example, when an Incident has High Impact, the system can check whether the **Assigned To** field has been filled. If the field is empty, the system prevents the record from being submitted and displays an appropriate error message. This ensures that important Incident records are not saved with missing information.

The **onCellEdit Client Script** controls changes made directly from the Incident list view. It can prevent users from modifying important fields such as **State** through direct list editing and instruct them to open the complete Incident record before making the change. This provides better control over important Incident updates and reduces accidental modifications.

Along with Client Scripts, **UI Policies** can be used to dynamically control form fields without requiring complex coding. UI Policies can make fields **mandatory, visible, or read-only** based on specific conditions. This allows the Incident form to respond automatically according to the information entered by the user.

Overall, the project demonstrates how ServiceNow scripting and configuration features can work together to create a more reliable

Incident Management process. The solution reduces manual work, prevents invalid data submission, improves user interaction, and maintains consistency across Incident records.

The project is also suitable for a **short-duration micro project** because the implementation is lightweight and does not require complex backend development. At the same time, it provides practical experience with important ServiceNow concepts such as **Client Scripts, UI Policies, form validation, field automation, list editing, and dynamic form behavior**.

Thus, the **Implement Client Script & UI Policy (Incident)** project provides a practical example of how ServiceNow customization can be used to improve **data integrity, automation, validation, security, and overall form usability** in an Incident Management system.